

To use Laravel Octane with Swoole we need to install this packages: composer require [laravel/Octane](#)

To install the PHP Swoole extension: sudo apt install [php8.4-openswoole](#)

Then, we need to activate the extension from the [php.ini](#) file.

=====

To start the Octane server: php artisan Octane:start --host 0.0.0.0

=====

To define a repeated task in the app service provider:

```
Octane::tick('caching-query', function () {
    Log::info('caching-query.', ['timestamp' => now()]);
    $time = hrtime(true);

    $count = Event::count();
    $eventsInfo = Event::ofType('INFO')->get();
    $eventsWarning = Event::ofType('WARNING')->get();
    $eventsAlert = Event::ofType('ALERT')->get();

    $time = (hrtime(true) - $time) / 1_000_000;
    $result = [
        'count' => $count,
        'eventsInfo'=> $eventsInfo,
        'eventsWarning' => $eventsWarning,
        'eventsAlert'=> $eventsAlert,
        'cached_time' => $time,
    ];
    Cache::store('Octane')->put('cached-result-tick', $result);
})->seconds(60)->immediate();
```

=====

Note (From me): if the user defines the [Octane concurrent](#) function inside the [Octane tick](#), an error will appear.

=====

To use the Octane concurrent function with the cache in Laravel:

```
Route::get('/httpcall/parallel', function () {
    $time = hrtime(true);
    [$sentenceJson, $nameJson] = Cache::store('Octane')->remember(
        'key-checking', 20, function () {
            return Octane::concurrently([
                fn() => Http::get('http://127.0.0.1:8000/api/sentence')->json(),
                fn() => Http::get('http://127.0.0.1:8000/api/name')->json(),
            ]);
        }
    );
    $time = hrtime(true) - $time;

    return response()->json(
        array_merge(
            $sentenceJson,
            $nameJson,
            ['time_ms' => $time / 1_000_000]
        )
    );
});
```

=====

To create Octane table for caching:

```
Route::get('/table-create', function () {
    // Getting the table instance
    $table = Octane::table('my-table');
    // looping 1..90 creating rows with fake() helper
    for ($i = 1; $i <= 90; $i++) {
        $table->set(
            $i,
            [
                'uuid' => fake()->uuid(),
                'name' => fake()->name(),
                'age' => fake()->numberBetween(18, 99),
                'value' => fake()->randomFloat(2, 0, 1000)
            ]
        );
    }
    return "Table created!";
});
```

And in the Octane config file:

```
'tables' => [
  'example:1000' => [
    'name' => 'string:1000',
    'votes' => 'int',
  ],
  'my-table:100' => [
    'uuid' => 'string:36',
    'name' => 'string:1000',
    'age' => 'int',
    'value' => 'float',
  ],
  'cache' => [
    'key' => 'string:256',
    'value' => 'string:2048576', // Increase this (e.g., 1MB)
    'expiration' => 'int:8',
  ],
  'request_context' => [
    'key' => 'string:256',
    'value' => 'string:2048576', // Increase this too
  ],
],

=====

Route::get('/table-get/', function () {
  $table = Octane::table('my-table');
  $row = $table->get(1);
  return $row;
});

-----

Route::get('/table-get-all', function () {
  $table = Octane::table('my-table');
  $rows = [];
  foreach ($table as $key => $value) {
    $rows[$key] = $table->get($key);
  }
  // adding as first row the table rows count
  $rows[0] = count($table);
  return $rows;
});

-----
```

```
Route::get('/metrics', function () {  
    $server = App::make(Swoole\Http\Server::class);  
    return $server->stats();  
});
```

=====